

การพัฒนา Model โดยใช้ Python และ Software Package สำหรับผู้พัฒนา Model

Module 6 - Regression (continued)

Asst. Prof. Dr. Santitham Prom-on

1. Load data

```
In [1]: !wget -O mtcars.csv https://www.dropbox.com/scl/fi/n0e5sbzbc1sf90n4kr72h/mtcars.
```

```
'wget' is not recognized as an internal or external command,  
operable program or batch file.  
'dl' is not recognized as an internal or external command,  
operable program or batch file.
```

```
In [2]: import pandas as pd  
  
mtcars = pd.read_csv('mtcars.csv', index_col=0)  
mtcars.head()
```

```
Out[2]:
```

	mpg	cyl	displacement	hp	drat	wt	qsec	vs	am	gear	carb
Mazda RX4	21.0	6	160.0	110	3.90	2.620	16.46	0	1	4	4
Mazda RX4 Wag	21.0	6	160.0	110	3.90	2.875	17.02	0	1	4	4
Datsun 710	22.8	4	108.0	93	3.85	2.320	18.61	1	1	4	1
Hornet 4 Drive	21.4	6	258.0	110	3.08	3.215	19.44	1	0	3	1
Hornet Sportabout	18.7	8	360.0	175	3.15	3.440	17.02	0	0	3	2

2. Assess correlations between variables

As a correlation matrix

```
In [3]: mtcars.corr()
```

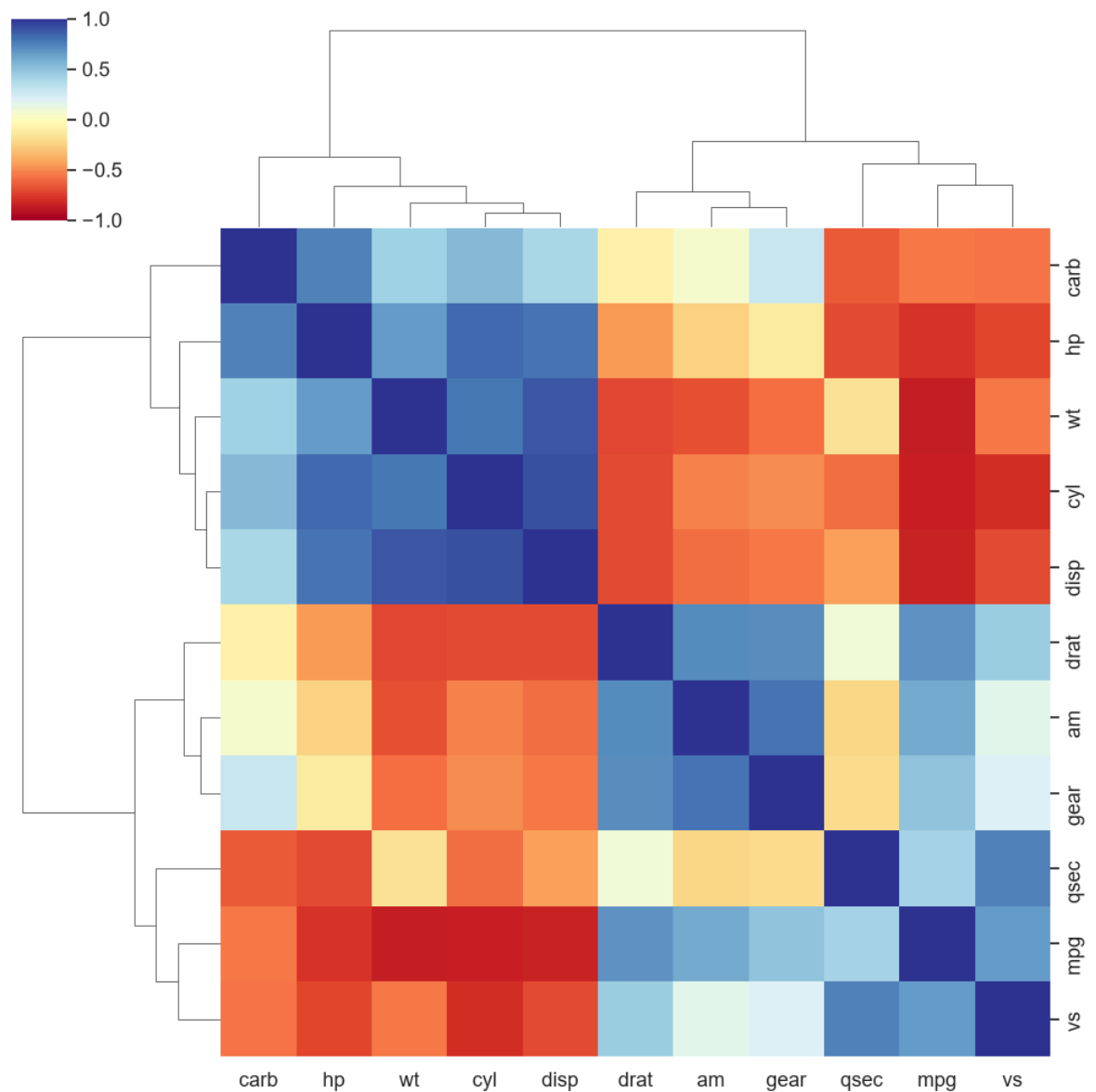
Out[3]:

	mpg	cyl	disp	hp	drat	wt	qsec	
mpg	1.000000	-0.852162	-0.847551	-0.776168	0.681172	-0.867659	0.418684	0.664039
cyl	-0.852162	1.000000	0.902033	0.832447	-0.699938	0.782496	-0.591242	-0.810812
disp	-0.847551	0.902033	1.000000	0.790949	-0.710214	0.887980	-0.433698	-0.710416
hp	-0.776168	0.832447	0.790949	1.000000	-0.448759	0.658748	-0.708223	-0.723097
drat	0.681172	-0.699938	-0.710214	-0.448759	1.000000	-0.712441	0.091205	0.440278
wt	-0.867659	0.782496	0.887980	0.658748	-0.712441	1.000000	-0.174716	-0.554916
qsec	0.418684	-0.591242	-0.433698	-0.708223	0.091205	-0.174716	1.000000	0.744535
vs	0.664039	-0.810812	-0.710416	-0.723097	0.440278	-0.554916	0.744535	1.000000
am	0.599832	-0.522607	-0.591227	-0.243204	0.712711	-0.692495	-0.229861	0.168374
gear	0.480285	-0.492687	-0.555569	-0.125704	0.699610	-0.583287	-0.212682	0.206351
carb	-0.550925	0.526988	0.394977	0.749812	-0.090790	0.427606	-0.656249	-0.569351

As a heatmap

```
In [4]: import seaborn as sns
import matplotlib.pyplot as plt
sns.set(font_scale=1.2)
```

```
In [5]: sns.clustermap(mtcars.corr(), cmap='RdYlBu', vmin=-1, vmax=1)
plt.show()
```

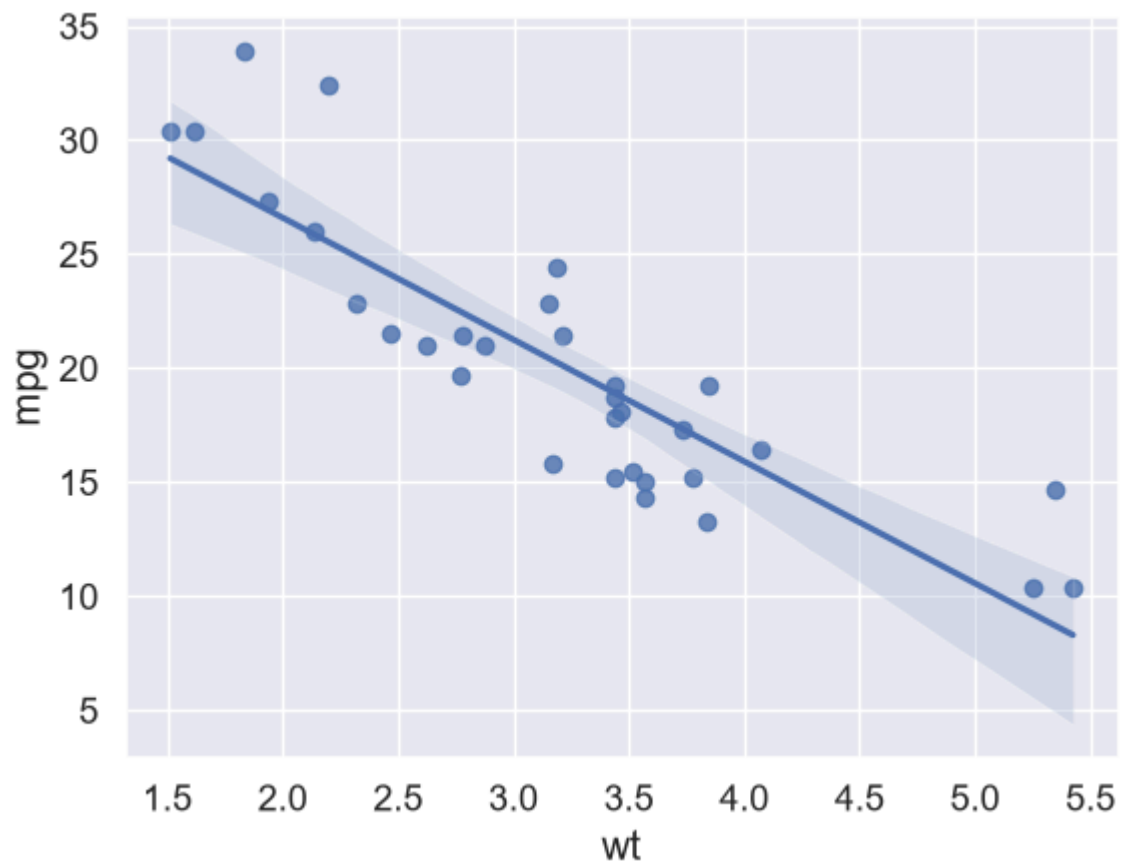


3. Visualize a linear relationship as a scatterplot

Negative relationship

```
In [6]: c = mtcars['mpg'].corr(mtcars['wt'])
print("Correlation = %0.2f"%mtcars['mpg'].corr(mtcars['wt']))
sns.regplot(data=mtcars, x='wt', y='mpg')
plt.show()
```

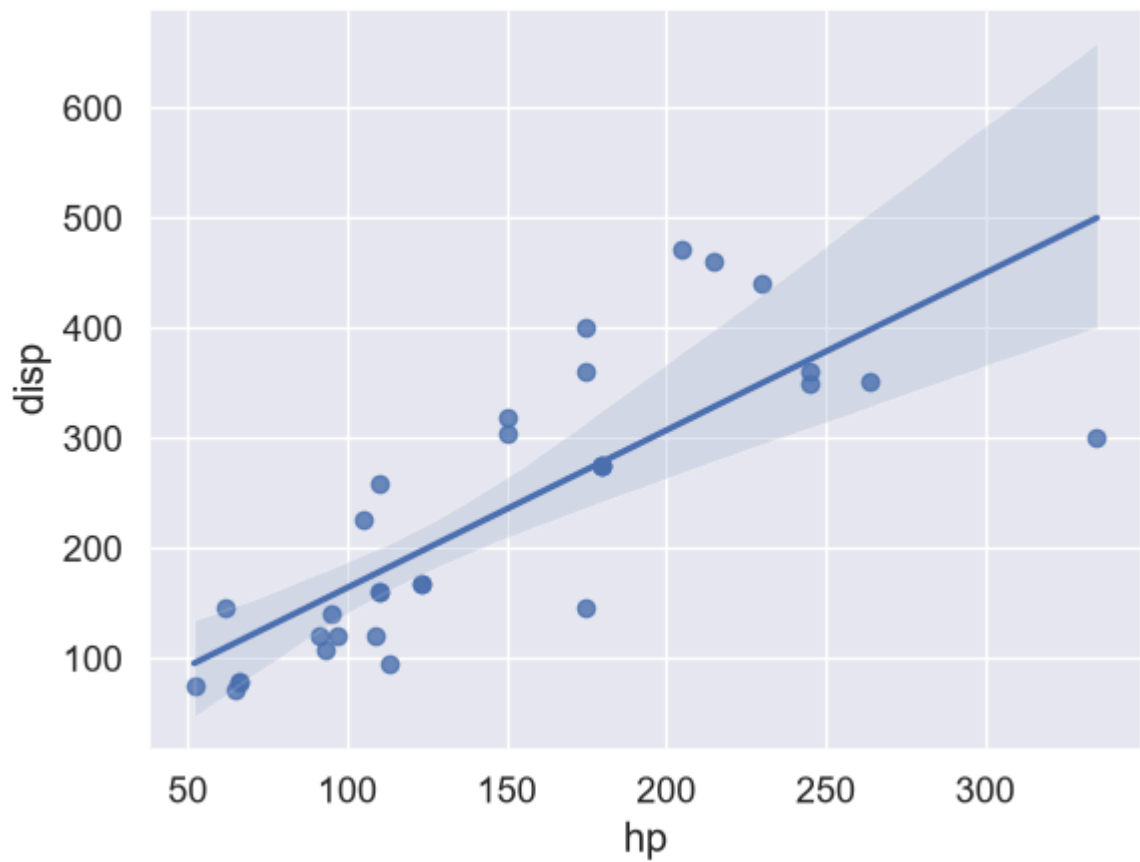
Correlation = -0.87



Positive relationship

```
In [7]: c = mtcars['hp'].corr(mtcars['disp'])
print("Correlation = %0.2f"%mtcars['hp'].corr(mtcars['disp']))
sns.regplot(data=mtcars, x='hp', y='disp')
plt.show()
```

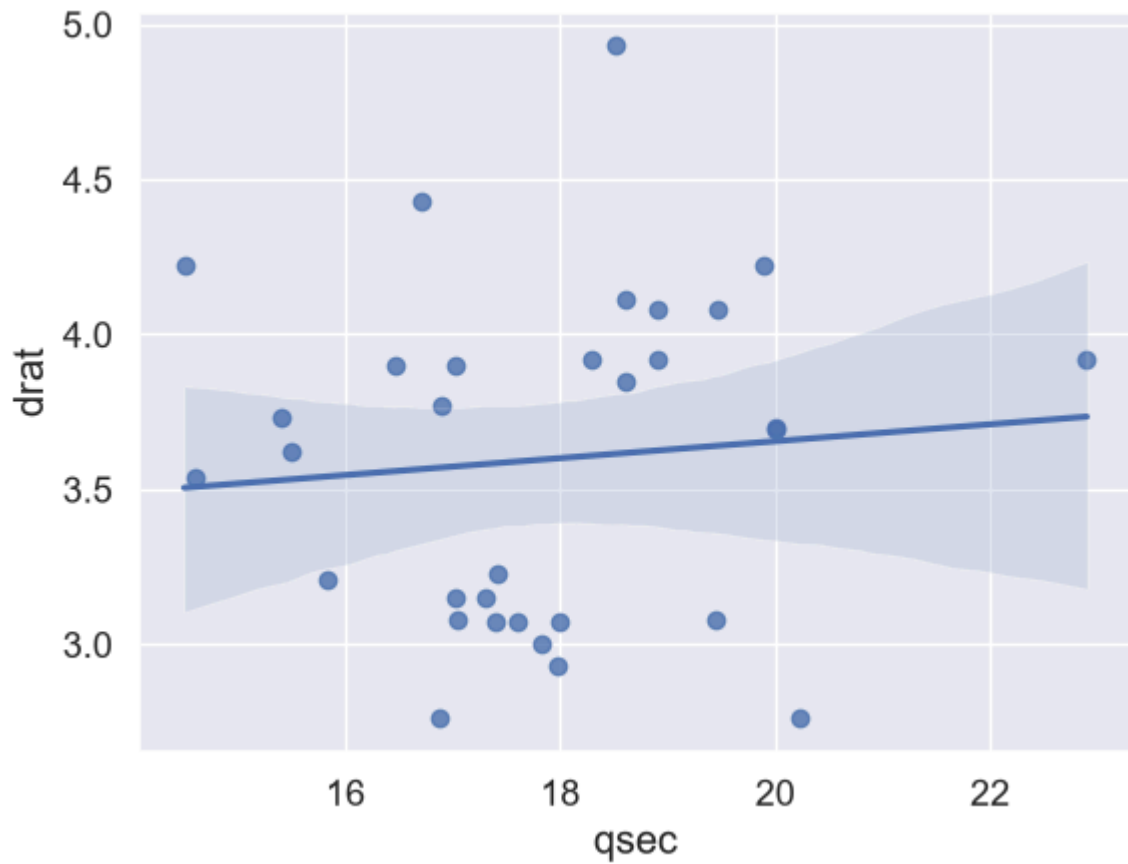
Correlation = 0.79



No relationship

```
In [8]: c = mtcars['qsec'].corr(mtcars['drat'])
print("Correlation = %0.2f"%mtcars['qsec'].corr(mtcars['drat']))
sns.regplot(data=mtcars, x='qsec', y='drat')
plt.show()
```

Correlation = 0.09



4. Linear regression

with statmodels

```
In [9]: from statsmodels.formula.api import ols

model = ols('mpg ~ wt',mtcars).fit()
model.summary()
```

Out[9]:

OLS Regression Results						
Dep. Variable:		mpg		R-squared:		0.753
Model:		OLS		Adj. R-squared:		0.745
Method:		Least Squares		F-statistic:		91.38
Date:		Fri, 28 Aug 2026		Prob (F-statistic):		1.29e-10
Time:		11:43:20		Log-Likelihood:		-80.015
No. Observations:		32		AIC:		164.0
Df Residuals:		30		BIC:		167.0
Df Model:		1				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
Intercept	37.2851	1.878	19.858	0.000	33.450	41.120
wt	-5.3445	0.559	-9.559	0.000	-6.486	-4.203
Omnibus:		2.988	Durbin-Watson:		1.252	
Prob(Omnibus):		0.225	Jarque-Bera (JB):		2.399	
Skew:		0.668	Prob(JB):		0.301	
Kurtosis:		2.877	Cond. No.		12.7	

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```
In [10]: model.predict(mtcars)
```

```
Out[10]: Mazda RX4                23.282611
Mazda RX4 Wag                   21.919770
Datsun 710                      24.885952
Hornet 4 Drive                  20.102650
Hornet Sportabout              18.900144
Valiant                         18.793255
Duster 360                     18.205363
Merc 240D                      20.236262
Merc 230                       20.450041
Merc 280                       18.900144
Merc 280C                      18.900144
Merc 450SE                     15.533127
Merc 450SL                     17.350247
Merc 450SLC                    17.083024
Cadillac Fleetwood             9.226650
Lincoln Continental            8.296712
Chrysler Imperial             8.718926
Fiat 128                       25.527289
Honda Civic                    28.653805
Toyota Corolla                 27.478021
Toyota Corona                  24.111004
Dodge Challenger               18.472586
AMC Javelin                    18.926866
Camaro Z28                     16.762355
Pontiac Firebird               16.735633
Fiat X1-9                      26.943574
Porsche 914-2                  25.847957
Lotus Europa                   29.198941
Ford Pantera L                 20.343151
Ferrari Dino                   22.480940
Maserati Bora                  18.205363
Volvo 142E                     22.427495
dtype: float64
```

with sci-kit learn

```
In [11]: X = mtcars.drop(columns='mpg')
y = mtcars['mpg']
```

```
In [12]: from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_df = pd.DataFrame(scaler.fit_transform(X), columns = X.columns)
```

```
In [13]: X_df
```

Out[13]:

	cyl	disp	hp	drat	wt	qsec	vs	am
0	-0.106668	-0.579750	-0.543655	0.576594	-0.620167	-0.789601	-0.881917	1.20894
1	-0.106668	-0.579750	-0.543655	0.576594	-0.355382	-0.471202	-0.881917	1.20894
2	-1.244457	-1.006026	-0.795570	0.481584	-0.931678	0.432823	1.133893	1.20894
3	-0.106668	0.223615	-0.543655	-0.981576	-0.002336	0.904736	1.133893	-0.827170
4	1.031121	1.059772	0.419550	-0.848562	0.231297	-0.471202	-0.881917	-0.827170
5	-0.106668	-0.046906	-0.617748	-1.589643	0.252064	1.348220	1.133893	-0.827170
6	1.031121	1.059772	1.456847	-0.734549	0.366285	-1.142114	-0.881917	-0.827170
7	-1.244457	-0.688779	-1.254944	0.177551	-0.028296	1.223135	1.133893	-0.827170
8	-1.244457	-0.737144	-0.765933	0.614599	-0.069830	2.871986	1.133893	-0.827170
9	-0.106668	-0.517448	-0.351014	0.614599	0.231297	0.256567	1.133893	-0.827170
10	-0.106668	-0.517448	-0.351014	0.614599	0.231297	0.597708	1.133893	-0.827170
11	1.031121	0.369533	0.493642	-1.000578	0.885470	-0.255145	-0.881917	-0.827170
12	1.031121	0.369533	0.493642	-1.000578	0.532424	-0.141432	-0.881917	-0.827170
13	1.031121	0.369533	0.493642	-1.000578	0.584343	0.085996	-0.881917	-0.827170
14	1.031121	1.977904	0.864106	-1.266608	2.110747	0.074625	-0.881917	-0.827170
15	1.031121	1.879533	1.012291	-1.133593	2.291423	-0.016346	-0.881917	-0.827170
16	1.031121	1.715580	1.234569	-0.696545	2.209392	-0.243774	-0.881917	-0.827170
17	-1.244457	-1.246216	-1.195670	0.918632	-1.056282	0.921793	1.133893	1.20894
18	-1.244457	-1.270809	-1.403130	2.533809	-1.663729	0.381652	1.133893	1.20894
19	-1.244457	-1.308518	-1.210489	1.184661	-1.435287	1.166278	1.133893	1.20894
20	-1.244457	-0.906835	-0.736296	0.196553	-0.781114	1.228820	1.133893	-0.827170
21	1.031121	0.715472	0.049086	-1.589643	0.314367	-0.556487	-0.881917	-0.827170
22	1.031121	0.600705	0.049086	-0.848562	0.226105	-0.312002	-0.881917	-0.827170
23	1.031121	0.977795	1.456847	0.253559	0.646645	-1.386598	-0.881917	-0.827170
24	1.031121	1.387676	0.419550	-0.981576	0.651837	-0.454145	-0.881917	-0.827170
25	-1.244457	-1.243757	-1.195670	0.918632	-1.331450	0.597708	1.133893	1.20894
26	-1.244457	-0.905195	-0.825207	1.583705	-1.118584	-0.653144	-0.881917	1.20894
27	-1.244457	-1.111775	-0.499199	0.329567	-1.769642	-0.539430	1.133893	1.20894
28	1.031121	0.985993	1.738399	1.184661	-0.049063	-1.903996	-0.881917	1.20894
29	-0.106668	-0.702714	0.419550	0.044536	-0.464411	-1.335427	-0.881917	1.20894
30	1.031121	0.576113	2.790515	-0.107481	0.366285	-1.847139	-0.881917	1.20894
31	-1.244457	-0.899457	-0.558473	0.975638	-0.454027	0.427138	1.133893	1.20894

```
In [14]: from sklearn.linear_model import LinearRegression
lm = LinearRegression()
lm.fit(X_df,y)
```

```
Out[14]: ▼ LinearRegression ⓘ ?
          ► Parameters
          ► Fitted attributes
```

```
In [15]: from sklearn.metrics import r2_score
r2_score(y_pred=lm.predict(X_df), y_true=y)
```

```
Out[15]: 0.8690157644777647
```

```
In [16]: pd.DataFrame(dict(Feature = X_df.columns,
                           Coefficient = lm.coef_))
```

```
Out[16]:
```

	Feature	Coefficient
0	cyl	-0.195890
1	disp	1.626723
2	hp	-1.449679
3	drat	0.414224
4	wt	-3.578015
5	qsec	1.444047
6	vs	0.157635
7	am	1.237765
8	gear	0.475951
9	carb	-0.317029

5. Poisson regression

```
In [17]: !wget -O mines.csv https://www.dropbox.com/scl/fi/y5wi9iqgyxjiwm4stdbx3/mines.cs
```

```
'wget' is not recognized as an internal or external command,
operable program or batch file.
'dl' is not recognized as an internal or external command,
operable program or batch file.
```

```
In [18]: import pandas as pd
mines = pd.read_csv('mines.csv')
mines.head()
```

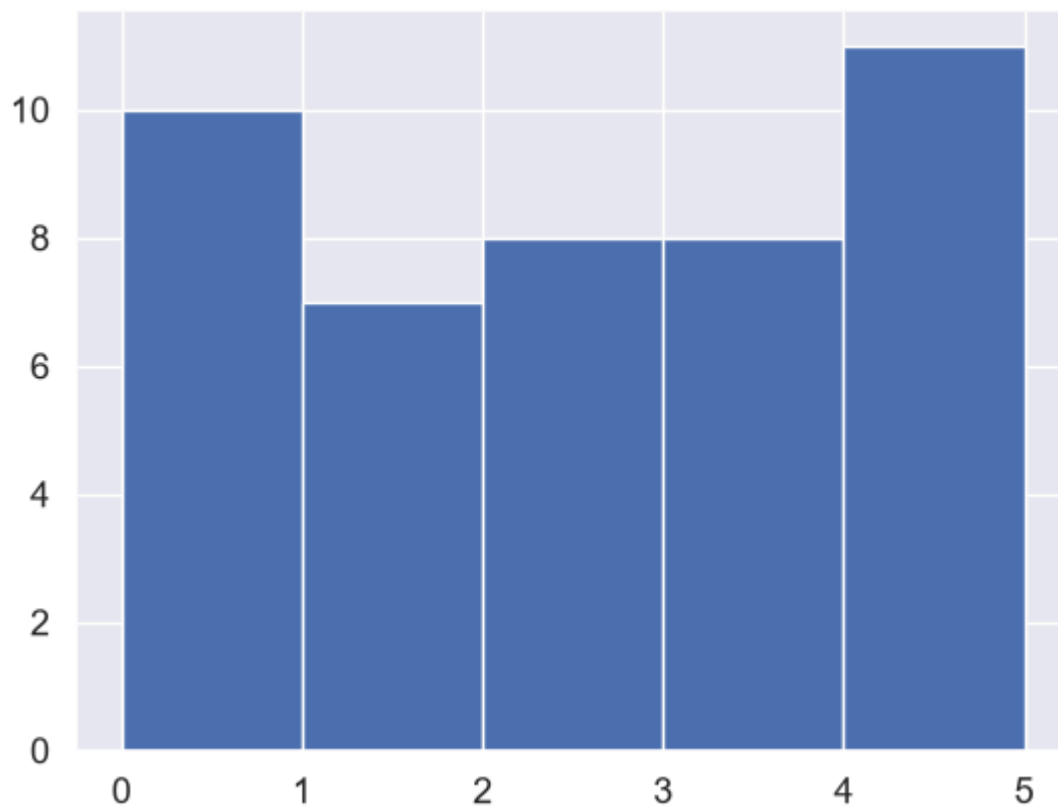
```
Out[18]:
```

	COUNT	INB	EXTRP	AHS	AGE
0	2	50	70	52	1.0
1	1	230	65	42	6.0
2	0	125	70	45	1.0
3	4	75	65	68	0.5
4	1	70	65	53	0.5

6. Visualize data distribution

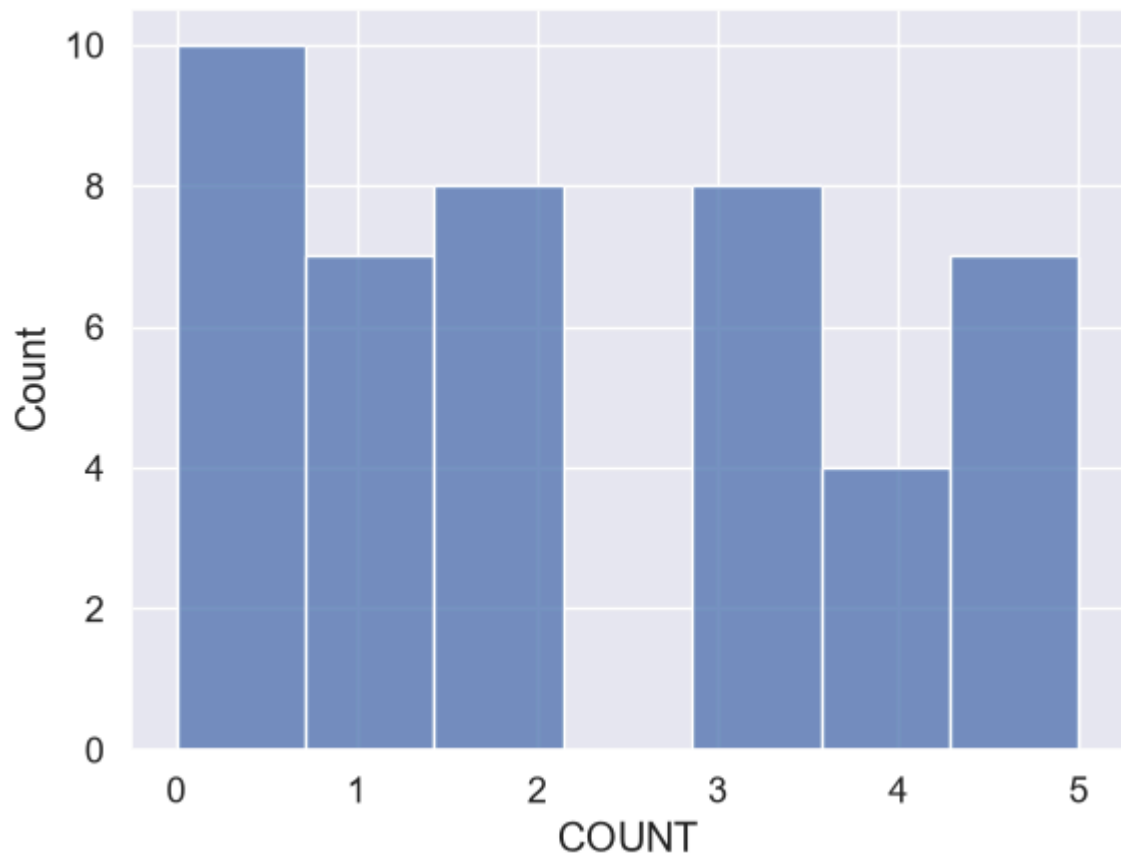
```
In [19]: mines['COUNT'].hist(bins=5)
```

```
Out[19]: <Axes: >
```



```
In [20]: import seaborn as sns
sns.histplot(mines['COUNT'])
```

```
Out[20]: <Axes: xlabel='COUNT', ylabel='Count'>
```



7. Poisson regression

with statmodels

```
In [21]: import statsmodels.api as sm  
from statsmodels.genmod.families import Poisson
```

```
In [22]: X = mines.drop(columns='COUNT')  
X_prep = sm.add_constant(X)  
y = mines['COUNT']  
model = sm.GLM(y, X_prep, family = Poisson()).fit()  
model.summary()
```

Out[22]:

Generalized Linear Model Regression Results

Dep. Variable:	COUNT	No. Observations:	44
Model:	GLM	Df Residuals:	39
Model Family:	Poisson	Df Model:	4
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-66.995
Date:	Fri, 28 Aug 2026	Deviance:	37.717
Time:	11:43:21	Pearson chi2:	35.8
No. Iterations:	5	Pseudo R-squ. (CS):	0.5713
Covariance Type:	nonrobust		

	coef	std err	z	P> z	[0.025	0.975]
const	-3.6097	1.028	-3.510	0.000	-5.625	-1.594
INB	-0.0014	0.001	-1.716	0.086	-0.003	0.000
EXTRP	0.0622	0.012	5.062	0.000	0.038	0.086
AHS	-0.0018	0.005	-0.346	0.729	-0.012	0.008
AGE	-0.0296	0.016	-1.816	0.069	-0.062	0.002

```
In [23]: y_res = model.predict(X_prep)
```

```
In [24]: from sklearn.metrics import mean_absolute_error
mean_absolute_error(y_true=y, y_pred=y_res)
```

Out[24]: 0.9307388957080917

Evaluation

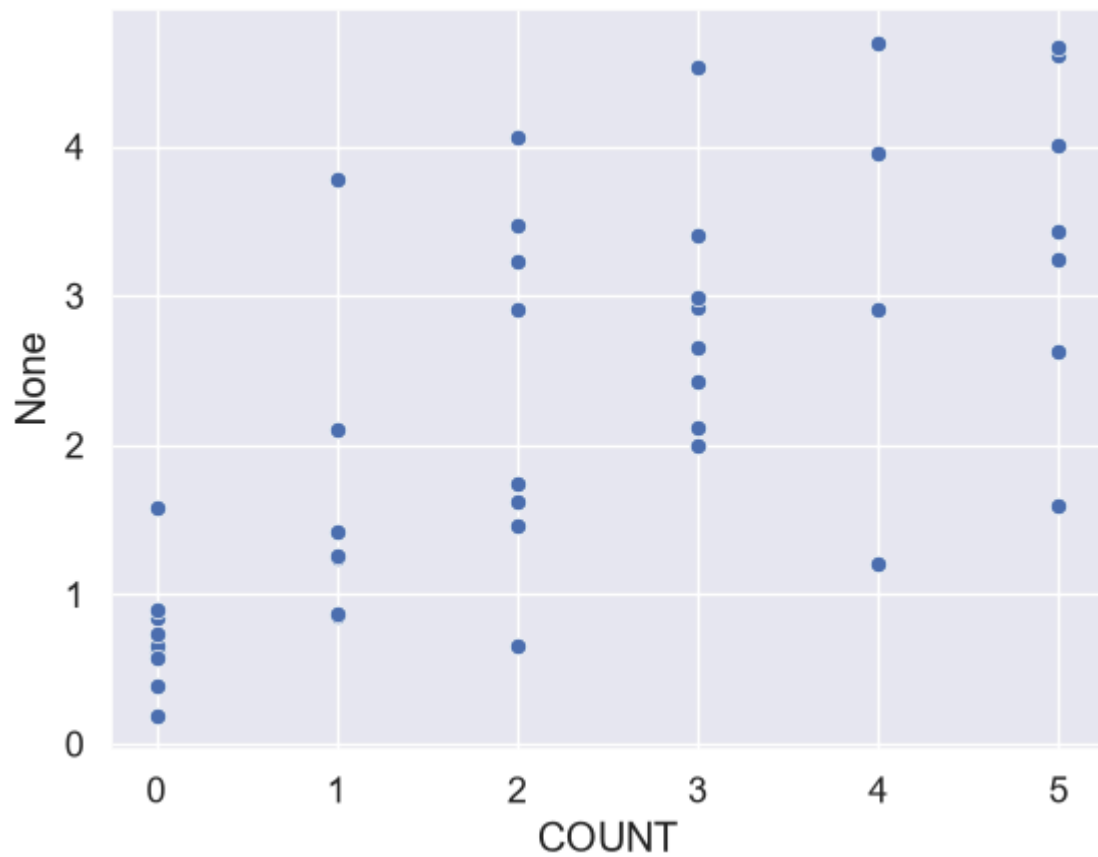
```
In [25]: from sklearn.metrics import mean_absolute_error
```

```
In [26]: mean_absolute_error(y_pred=model.predict(X_prep), y_true=y)
```

Out[26]: 0.9307388957080917

```
In [27]: sns.scatterplot(x=y, y=model.predict(X_prep))
```

Out[27]: <Axes: xlabel='COUNT', ylabel='None'>



Compare to linear regression

MAE of Poisson Regression = 0.9307

```
In [28]: X_prep = sm.add_constant(X)
lm = sm.OLS(y, X_prep).fit()
lm.summary()
```

Out[28]:

OLS Regression Results						
Dep. Variable:		COUNT		R-squared:		0.530
Model:		OLS		Adj. R-squared:		0.482
Method:		Least Squares		F-statistic:		11.00
Date:		Fri, 28 Aug 2026		Prob (F-statistic):		4.55e-06
Time:		11:43:21		Log-Likelihood:		-69.941
No. Observations:		44		AIC:		149.9
Df Residuals:		39		BIC:		158.8
Df Model:		4				
Covariance Type:		nonrobust				
	coef	std err	t	P> t	[0.025	0.975]
const	-4.5797	1.379	-3.321	0.002	-7.369	-1.790
INB	-0.0020	0.001	-1.524	0.136	-0.005	0.001
EXTRP	0.1042	0.017	6.119	0.000	0.070	0.139
AHS	-0.0077	0.009	-0.901	0.373	-0.025	0.010
AGE	-0.0486	0.028	-1.725	0.092	-0.106	0.008
Omnibus:		1.311	Durbin-Watson:		2.526	
Prob(Omnibus):		0.519	Jarque-Bera (JB):		1.056	
Skew:		0.375	Prob(JB):		0.590	
Kurtosis:		2.877	Cond. No.		1.77e+03	

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.77e+03. This might indicate that there are strong multicollinearity or other numerical problems.

```
In [29]: y_res = lm.predict(X_prep)
         mean_absolute_error(y, y_res)
```

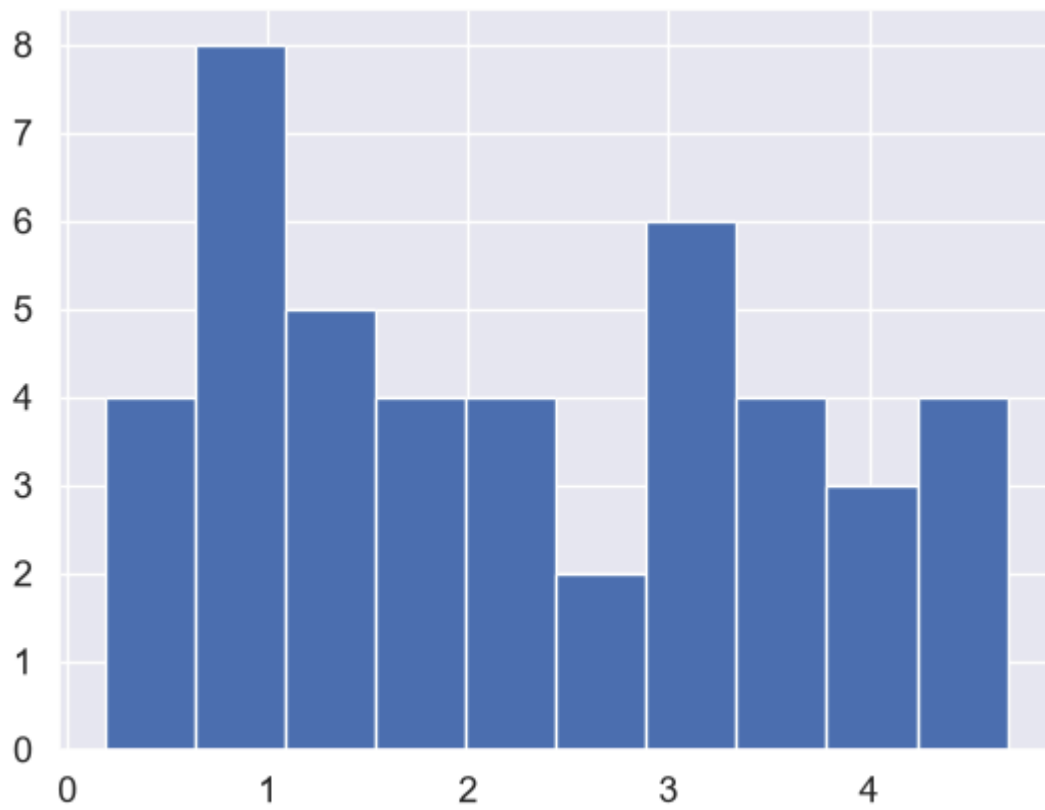
Out[29]: 0.9366988953509329

```
In [30]: y_res
```

```
Out[30]: 0      2.170637
         1      1.128059
         2      2.076445
         3      1.501468
         4      1.626576
         5      2.089919
         6      1.021765
         7      0.545242
         8      3.672588
         9      3.152231
        10      1.132050
        11      3.900345
        12      1.942372
        13      3.367701
        14      3.223064
        15      3.806386
        16      3.869613
        17      3.262442
        18      0.595756
        19      3.490417
        20      3.518025
        21      1.829621
        22      2.344900
        23      2.746063
        24      1.462799
        25      3.757984
        26     -1.360553
        27      2.533680
        28      2.038730
        29      0.956509
        30      3.157593
        31      3.278133
        32      3.893867
        33     -0.244533
        34      0.536512
        35      3.142633
        36      0.919443
        37      1.008225
        38      2.949301
        39      3.095132
        40      2.581973
        41      2.894245
        42      0.322657
        43      3.061987
dtype: float64
```

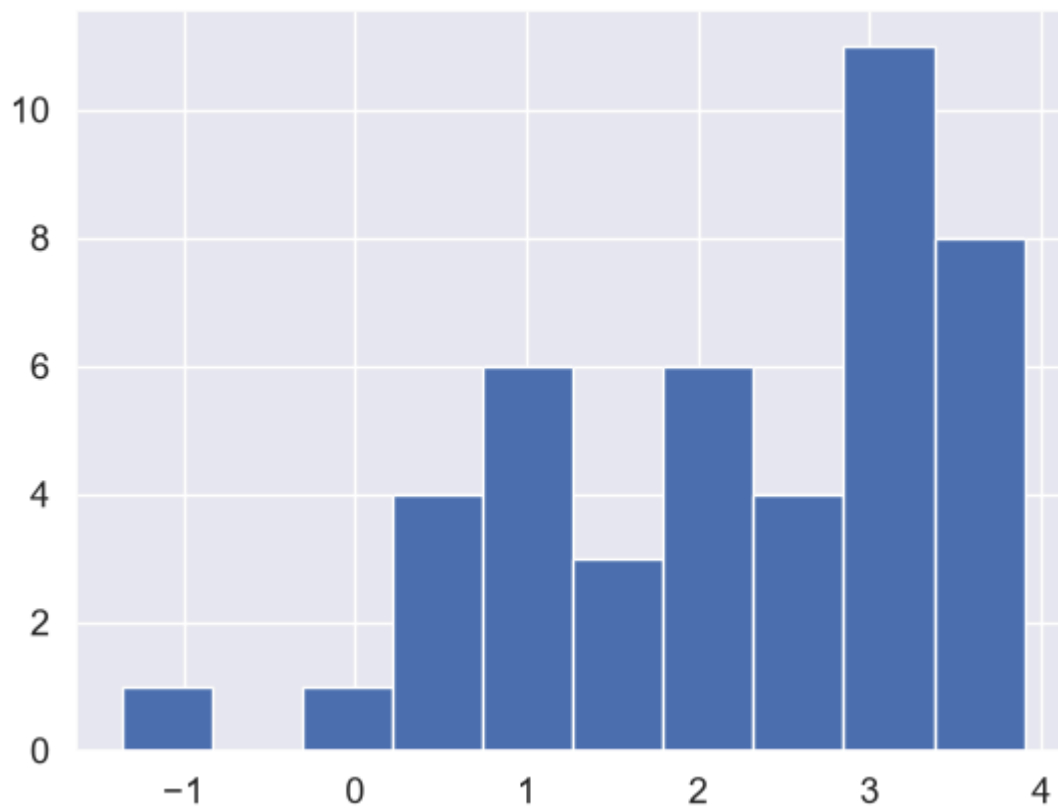
```
In [31]: model.predict(X_prep).hist()
```

```
Out[31]: <Axes: >
```



```
In [32]: lm.predict(X_prep).hist()
```

```
Out[32]: <Axes: >
```



Activity

Use the cancer.xls data and perform Poisson regression analysis

```
In [33]: !wget -O cancer.xls https://www.dropbox.com/scl/fi/liwhrsk2voczh7jmajrts/cancer.
```

'wget' is not recognized as an internal or external command,
operable program or batch file.
'dl' is not recognized as an internal or external command,
operable program or batch file.

```
In [34]: # =====
# Activity: Poisson regression analysis on cancer.xls
# 67070501015 Naphat Utabuawong
# =====

import pandas as pd
import statsmodels.api as sm
from statsmodels.genmod.families import Poisson
from sklearn.preprocessing import MinMaxScaler
from sklearn.metrics import mean_absolute_error

# 1) Load the data (the table lives on 'Sheet2')
cancer = pd.read_excel('cancer.xls', sheet_name='Sheet2', index_col='index')
display(cancer.head())

# 2) Prepare predictors
# age & smoke are categorical -> one-hot encode (drop_first avoids the dummy
# rename smoke 'no' -> '0_no' so the non-smoker group becomes the reference L
# pop is the population size (numeric)
cancer.replace({'smoke': {'no': '0_no'}}, inplace=True)
cancer_cat = pd.get_dummies(cancer[['age', 'smoke']], drop_first=True)
cancer_prep = pd.concat([cancer_cat, cancer['pop']], axis=1)

# 3) Scale features to [0, 1]
cancer_prep_1 = pd.DataFrame(
    MinMaxScaler().fit_transform(cancer_prep),
    columns=cancer_prep.columns,
)

# 4) Fit a Poisson regression (GLM with a log link) on the death counts
y = cancer['dead'].reset_index(drop=True)
X = sm.add_constant(cancer_prep_1)
model = sm.GLM(y, X, family=Poisson()).fit()
print(model.summary())

# 5) Evaluate the fit
y_pred = model.predict(X)
print()
print("MAE:", mean_absolute_error(y_true=y, y_pred=y_pred))
```

	age	smoke	pop	dead
index				
1	40-44	no	656	18
2	45-59	no	359	22
3	50-54	no	249	19
4	55-59	no	632	55
5	60-64	no	1067	117

Generalized Linear Model Regression Results

```

=====
Dep. Variable:          dead    No. Observations:          36
Model:                  GLM     Df Residuals:              23
Model Family:           Poisson Df Model:                  12
Link Function:           Log    Scale:                    1.0000
Method:                  IRLS   Log-Likelihood:          -412.46
Date:                    Fri, 28 Aug 2026 Deviance:            584.90
Time:                    11:43:22 Pearson chi2:          508.
No. Iterations:          6      Pseudo R-squ. (CS):      1.000
Covariance Type:         nonrobust
=====

```

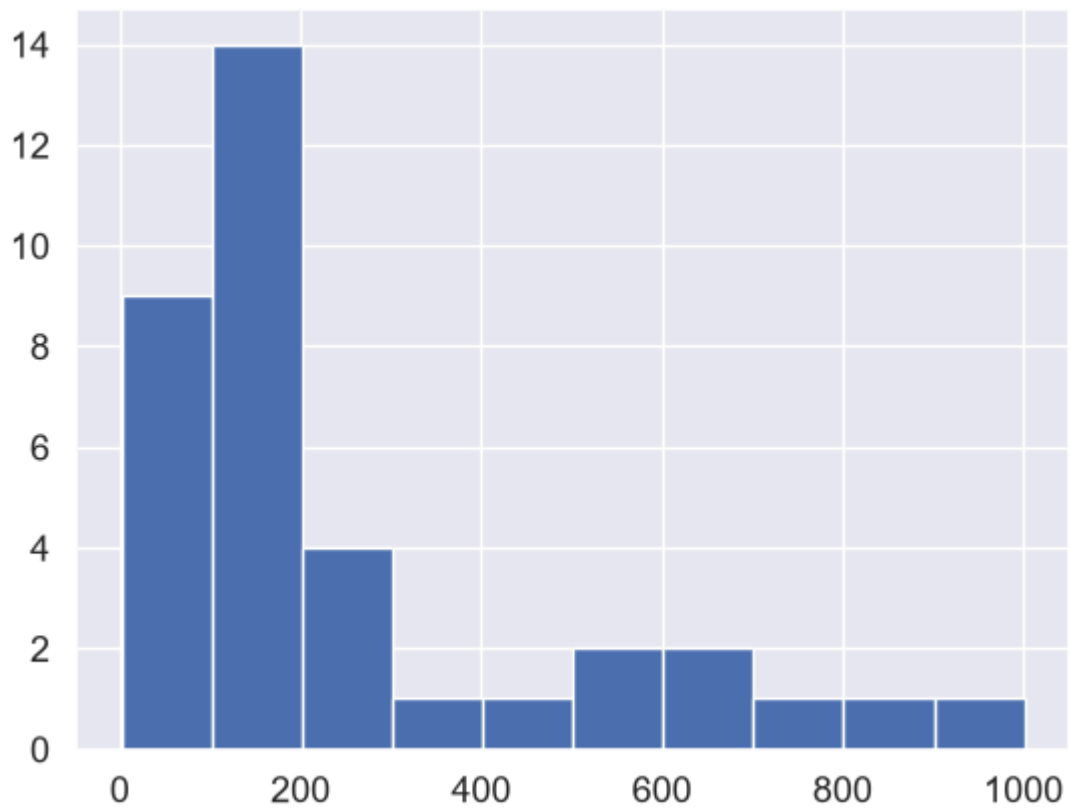
	coef	std err	z	P> z	[0.025
0.975]					

const	2.6013	0.075	34.887	0.000	2.455
2.747					
age_45-59	0.6242	0.083	7.556	0.000	0.462
0.786					
age_50-54	1.0038	0.082	12.256	0.000	0.843
1.164					
age_55-59	1.3818	0.065	21.174	0.000	1.254
1.510					
age_60-64	1.5012	0.065	23.044	0.000	1.374
1.629					
age_65-69	2.1366	0.064	33.517	0.000	2.012
2.262					
age_70-74	2.3557	0.072	32.939	0.000	2.215
2.496					
age_75-79	2.2041	0.079	27.817	0.000	2.049
2.359					
age_80+	1.9506	0.085	23.084	0.000	1.785
2.116					
smoke_cigarPipeOnly	0.2114	0.047	4.520	0.000	0.120
0.303					
smoke_cigaretteOnly	0.7818	0.044	17.868	0.000	0.696
0.868					
smoke_cigarettePlus	0.5030	0.057	8.821	0.000	0.391
0.615					
pop	2.4840	0.099	25.099	0.000	2.290
2.678					
=====					
=====					

MAE: 42.53348516800903

In [35]: cancer['dead'].hist()

Out[35]: <Axes: >



```
In [36]: cancer['smoke'].value_counts()
```

```
Out[36]: smoke
0_no      9
cigarPipeOnly  9
cigarettePlus  9
cigaretteOnly  9
Name: count, dtype: int64
```

```
In [37]: cancer.replace({'smoke':{'no':'0_no'}}, inplace=True)
```

```
In [38]: cancer_cat = pd.get_dummies(cancer[['age', 'smoke']], drop_first=True)
cancer_cat.head()
```

```
Out[38]:
```

	age_45-59	age_50-54	age_55-59	age_60-64	age_65-69	age_70-74	age_75-79	age_80+	smoke_0_no
index									
1	False	False	False	False	False	False	False	False	False
2	True	False	False	False	False	False	False	False	False
3	False	True	False	False	False	False	False	False	False
4	False	False	True	False	False	False	False	False	False
5	False	False	False	True	False	False	False	False	False

```
In [39]: cancer_prep = pd.concat([cancer_cat, cancer['pop']], axis=1)
```

```
In [40]: from sklearn.preprocessing import MinMaxScaler
mms = MinMaxScaler()
```

```
cancer_prep_1 = pd.DataFrame(mms.fit_transform(cancer_prep), columns=cancer_prep)
cancer_prep_1.head()
```

Out[40]:

	age_45-59	age_50-54	age_55-59	age_60-64	age_65-69	age_70-74	age_75-79	age_80+	smoke_cig
0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	
2	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	
3	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	
4	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	

In [41]:

cancer['dead']

Out[41]:

index

1	18
2	22
3	19
4	55
5	117
6	170
7	179
8	120
9	120
10	2
11	4
12	3
13	38
14	113
15	173
16	212
17	243
18	253
19	149
20	169
21	193
22	576
23	1001
24	901
25	613
26	337
27	189
28	124
29	140
30	187
31	514
32	778
33	689
34	432
35	214
36	63

Name: dead, dtype: int64

In [42]:

```
model = sm.GLM(cancer['dead'], reset_index(drop=True),
               sm.add_constant(cancer_prep_1),
```

```
family = Poisson()).fit()
model.summary()
```

Out[42]:

Generalized Linear Model Regression Results

Dep. Variable:	dead	No. Observations:	36
Model:	GLM	Df Residuals:	23
Model Family:	Poisson	Df Model:	12
Link Function:	Log	Scale:	1.0000
Method:	IRLS	Log-Likelihood:	-412.46
Date:	Fri, 28 Aug 2026	Deviance:	584.90
Time:	11:43:22	Pearson chi2:	508.
No. Iterations:	6	Pseudo R-squ. (CS):	1.000
Covariance Type:	nonrobust		

	coef	std err	z	P> z	[0.025	0.975]
const	2.6013	0.075	34.887	0.000	2.455	2.747
age_45-59	0.6242	0.083	7.556	0.000	0.462	0.786
age_50-54	1.0038	0.082	12.256	0.000	0.843	1.164
age_55-59	1.3818	0.065	21.174	0.000	1.254	1.510
age_60-64	1.5012	0.065	23.044	0.000	1.374	1.629
age_65-69	2.1366	0.064	33.517	0.000	2.012	2.262
age_70-74	2.3557	0.072	32.939	0.000	2.215	2.496
age_75-79	2.2041	0.079	27.817	0.000	2.049	2.359
age_80+	1.9506	0.085	23.084	0.000	1.785	2.116
smoke_cigarPipeOnly	0.2114	0.047	4.520	0.000	0.120	0.303
smoke_cigaretteOnly	0.7818	0.044	17.868	0.000	0.696	0.868
smoke_cigarettePlus	0.5030	0.057	8.821	0.000	0.391	0.615
pop	2.4840	0.099	25.099	0.000	2.290	2.678

In [43]:

```
model.predict(sm.add_constant(cancer_prep_1))
```

```
Out[43]: 0      17.014227
         1      28.061053
         2      39.176018
         3      67.078251
         4      90.624064
         5     159.360220
         6     180.317513
         7     136.334058
         8     102.034593
         9      16.983526
        10      31.167641
        11      45.442675
        12      74.349106
        13     102.094869
        14     201.189285
        15     237.742306
        16     191.360037
        17     140.670556
        18     141.692760
        19     141.412588
        20     150.344691
        21     600.931821
        22    1199.219114
        23      914.734227
        24     526.974283
        25     278.894174
        26     173.796342
        27     117.309486
        28     134.358717
        29     167.036615
        30     440.640822
        31     617.061953
        32     657.716267
        33     490.965897
        34     307.411732
        35     208.498509
dtype: float64
```

8. Survival analysis

For this example, we will be investigating the lifetimes of political leaders around the world. A political leader, in this case, is defined by a single individual's time in office who controls the ruling regime. This political leader could be an elected president, unelected dictator, monarch, etc. The birth event is the start of the individual's tenure, and the death event is the retirement of the individual. Censoring can occur if they are a) still in offices at the time of dataset compilation (2008), or b) die while in power (this includes assassinations).

For example, the Bush regime began in 2000 and officially ended in 2008 upon his retirement, thus this regime's lifespan was eight years, and there was a "death" event observed. On the other hand, the JFK regime lasted 2 years, from 1961 and 1963, and the regime's official death event was not observed – JFK died before his official retirement.

```
In [44]: !pip install lifelines
```

Requirement already satisfied: lifelines in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (0.30.3)

Requirement already satisfied: numpy>=1.14.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (2.5.2)

Requirement already satisfied: scipy>=1.7.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (1.18.1)

Requirement already satisfied: pandas<3.0,>=2.1 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (2.3.3)

Requirement already satisfied: matplotlib>=3.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (3.11.1)

Requirement already satisfied: autograd>=1.5 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (1.9.1)

Requirement already satisfied: autograd-gamma>=0.3 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (0.5.0)

Requirement already satisfied: formulaic>=0.2.2 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from lifelines) (1.2.2)

Requirement already satisfied: python-dateutil>=2.8.2 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from pandas<3.0,>=2.1->lifelines) (2.9.0.post0)

Requirement already satisfied: pytz>=2020.1 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from pandas<3.0,>=2.1->lifelines) (2026.3.post1)

Requirement already satisfied: tzdata>=2022.7 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from pandas<3.0,>=2.1->lifelines) (2026.3)

Requirement already satisfied: interface-meta>=1.2.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from formulaic>=0.2.2->lifelines) (2.0.1)

Requirement already satisfied: narwhals>=1.17 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from formulaic>=0.2.2->lifelines) (2.25.0)

Requirement already satisfied: typing-extensions>=4.2.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from formulaic>=0.2.2->lifelines) (4.16.0)

Requirement already satisfied: wrapt>=1.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from formulaic>=0.2.2->lifelines) (2.3.0)

Requirement already satisfied: contourpy>=1.0.1 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (1.3.3)

Requirement already satisfied: cycycler>=0.10 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (0.12.1)

Requirement already satisfied: fonttools>=4.28.2 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (4.63.0)

Requirement already satisfied: kiwisolver>=1.3.1 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (1.5.0)

Requirement already satisfied: packaging>=20.0 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (26.3)

Requirement already satisfied: pillow>=9 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (12.3.0)

Requirement already satisfied: pyparsing>=3 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from matplotlib>=3.0->lifelines) (3.3.2)

Requirement already satisfied: six>=1.5 in C:\Users\putte\Documents\GitHub\LabML\venv\Lib\site-packages (from python-dateutil>=2.8.2->pandas<3.0,>=2.1->lifelines) (1.17.0)

In [45]: `from lifelines.datasets import load_dd`

```
data = load_dd()
data.head()
```

Out[45]:

	ctryname	cowcode2	politycode	un_region_name	un_continent_name	ehead
0	Afghanistan	700	700.0	Southern Asia	Asia	Mohammad Zahir Shah
1	Afghanistan	700	700.0	Southern Asia	Asia	Sardar Mohammad Daoud
2	Afghanistan	700	700.0	Southern Asia	Asia	Mohammad Zahir Shah
3	Afghanistan	700	700.0	Southern Asia	Asia	Sardar Mohammad Daoud
4	Afghanistan	700	700.0	Southern Asia	Asia	Nur Mohammad Taraki

In [46]:

data.shape

Out[46]: (1808, 12)

Let's take a look at our country.

In [47]:

data[data['ctryname'] == 'Thailand']

Out[47]:

	ctryname	cowcode2	politycode	un_region_name	un_continent_name	
1602	Thailand	800	800.0	South-Eastern Asia	Asia	Thamrong
1603	Thailand	800	800.0	South-Eastern Asia	Asia	Phibuls
1604	Thailand	800	800.0	South-Eastern Asia	Asia	Thanom K
1605	Thailand	800	800.0	South-Eastern Asia	Asia	Sar
1606	Thailand	800	800.0	South-Eastern Asia	Asia	Thanom K
1607	Thailand	800	800.0	South-Eastern Asia	Asia	Sanya T
1608	Thailand	800	800.0	South-Eastern Asia	Asia	Ku
1609	Thailand	800	800.0	South-Eastern Asia	Asia	
1610	Thailand	800	800.0	South-Eastern Asia	Asia	C
1611	Thailand	800	800.0	South-Eastern Asia	Asia	C
1612	Thailand	800	800.0	South-Eastern Asia	Asia	Prem Ti
1613	Thailand	800	800.0	South-Eastern Asia	Asia	Cl
1614	Thailand	800	800.0	South-Eastern Asia	Asia	
1615	Thailand	800	800.0	South-Eastern Asia	Asia	Chu
1616	Thailand	800	800.0	South-Eastern Asia	Asia	Banharn Si
1617	Thailand	800	800.0	South-Eastern Asia	Asia	Yon
1618	Thailand	800	800.0	South-Eastern Asia	Asia	Chu
1619	Thailand	800	800.0	South-Eastern Asia	Asia	Thaksin !
1620	Thailand	800	800.0	South-Eastern Asia	Asia	Surayud
1621	Thailand	800	800.0	South-Eastern Asia	Asia	Abhi:

```
In [48]: from lifelines import KaplanMeierFitter
kmf = KaplanMeierFitter()

T = data["duration"]
E = data["observed"]

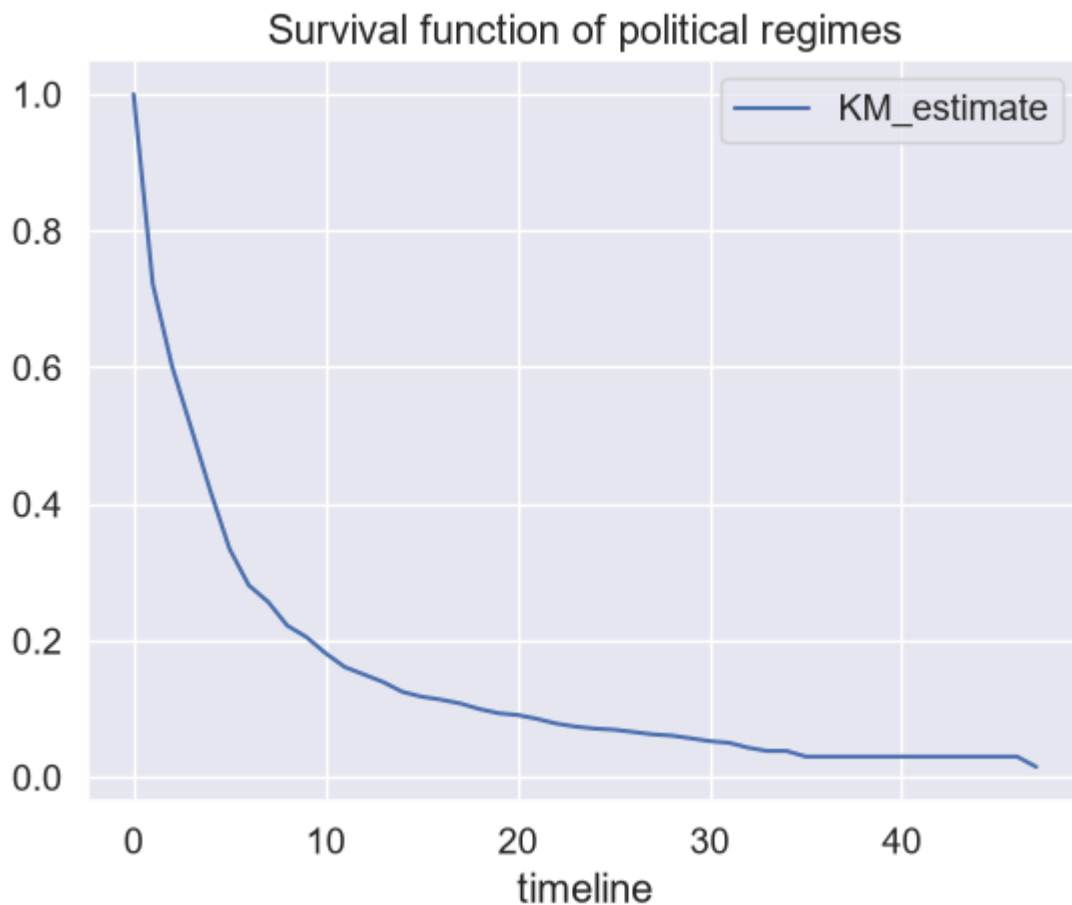
kmf.fit(T, event_observed=E)
```

```
Out[48]: <lifelines.KaplanMeierFitter:"KM_estimate", fitted with 1808 total observation
s, 340 right-censored observations>
```

```
In [49]: kmf.survival_function_at_times(10)
```

```
Out[49]: 10    0.18135
         Name: KM_estimate, dtype: float64
```

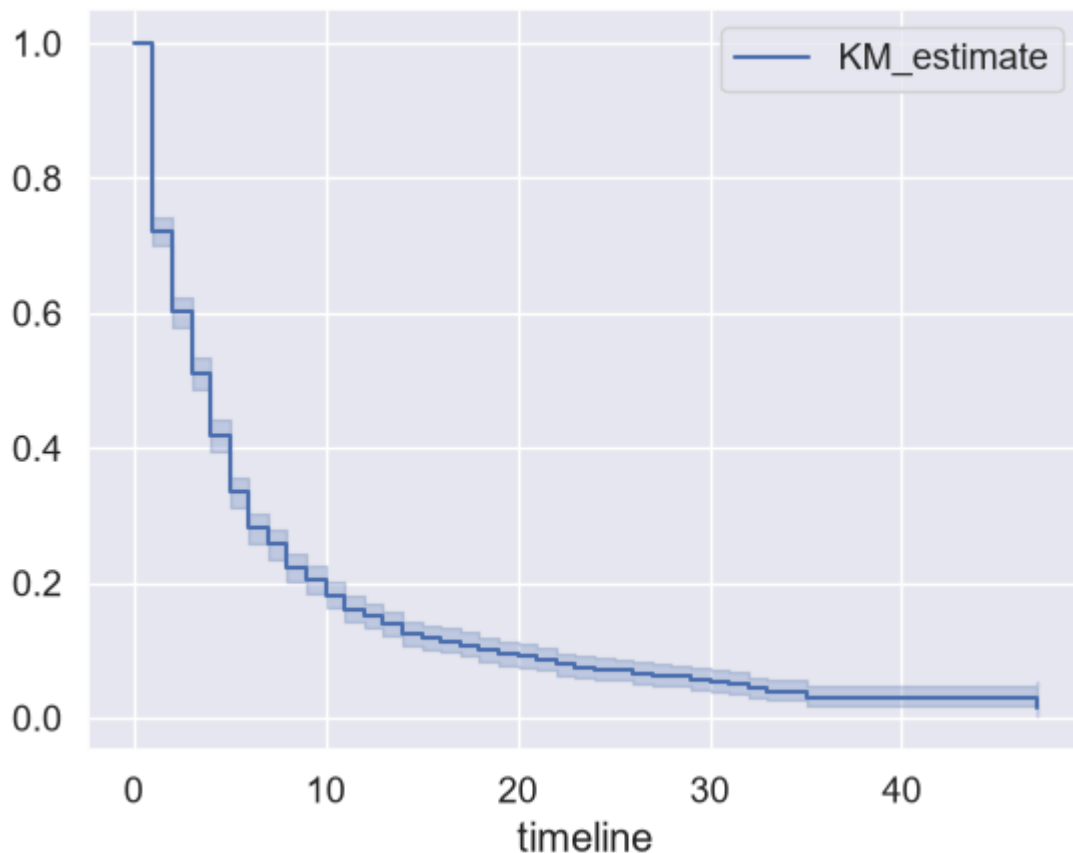
```
In [50]: import matplotlib.pyplot as plt
kmf.survival_function_.plot()
plt.title('Survival function of political regimes');
```



How do we interpret this? The y-axis represents the probability a leader is still around after t years, where t years is on the x-axis. We see that very few leaders make it past 20 years in office.

```
In [51]: kmf.plot()
```

```
Out[51]: <Axes: xlabel='timeline'>
```



The median time in office, which defines the point in time where on average 1/2 of the population has expired, is a property:

```
In [52]: kmf.median_survival_time_
```

```
Out[52]: np.float64(4.0)
```

Interesting that it is only four years. That means, around the world, elected leaders have a 50% chance of cessation in four years or less!

Plot by segment (regimes)

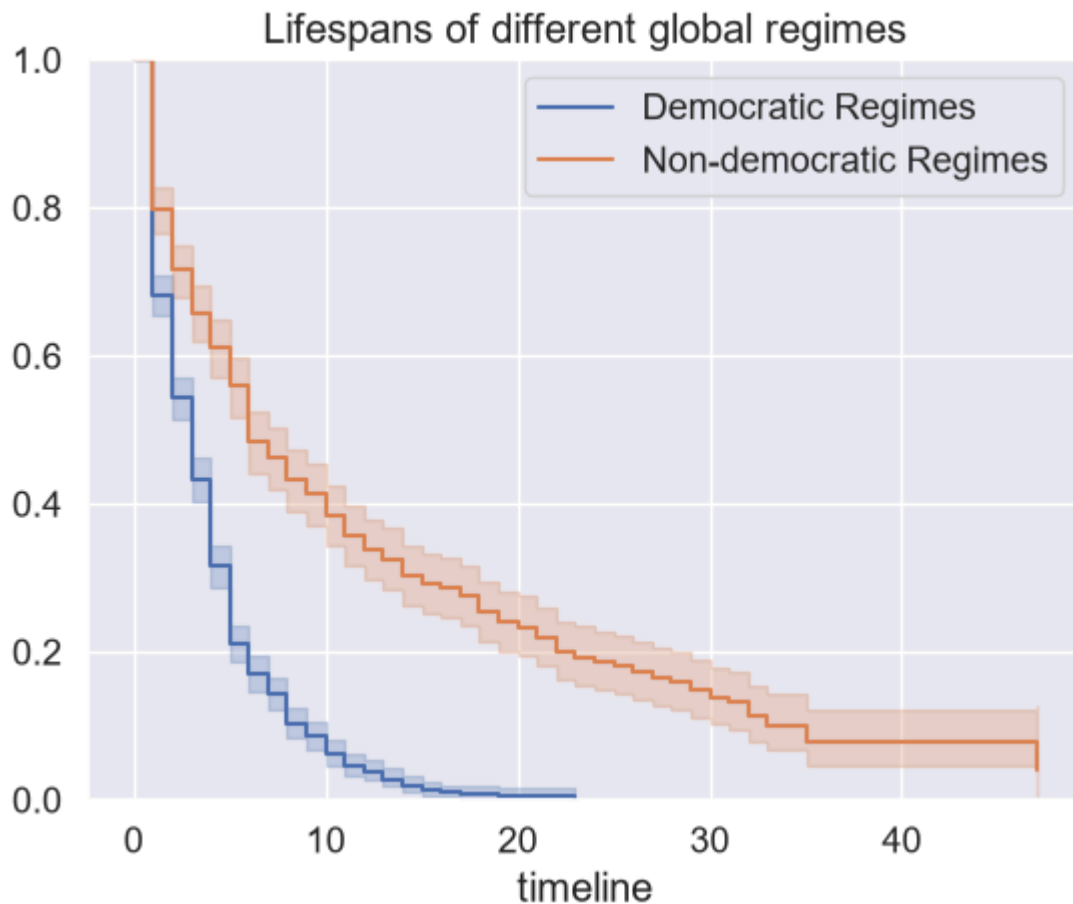
Let's segment on democratic regimes vs non-democratic regimes. Calling plot on either the estimate itself or the fitter object will return an axis object, that can be used for plotting further estimates:

```
In [53]: ax = plt.subplot(111)

dem = (data["democracy"] == "Democracy")

kmf.fit(T[dem], event_observed=E[dem], label="Democratic Regimes")
kmf.plot(ax=ax)
kmf.fit(T[~dem], event_observed=E[~dem], label="Non-democratic Regimes")
kmf.plot(ax=ax)

plt.ylim(0, 1);
plt.title("Lifespans of different global regimes");
```



Mathematical Notations

Survival function

We will assume for now that T is a continuous random variable with probability density function (p.d.f.)

$$f(t)$$

and cumulative distribution function (c.d.f.)

$$F(t) = \Pr(T < t),$$

giving the probability that the event has occurred by duration t .

$$S(t) = \Pr\{T \geq t\} = 1 - F(t) = \int_t^{\infty} f(x)dx,$$

Hazard function

An alternative characterization of the distribution of T is given by the hazard function, or instantaneous rate of occurrence of the event, defined as

$$\lambda(t) = \lim_{dt \rightarrow 0} \frac{\Pr\{t \leq T < t + dt | T \geq t\}}{dt}.$$

9. Cox's proportional hazard model

The idea behind Cox's proportional hazard model is that the log-hazard of an individual is a linear function of their static covariates and a population-level baseline hazard that changes over time. Mathematically:

$$\underbrace{h(t|x)}_{\text{hazard}} = \underbrace{b_0(t)}_{\text{baseline hazard}} \underbrace{\exp\left(\sum_{i=1}^n b_i(x_i - \bar{x}_i)\right)}_{\text{partial hazard}}^{\text{log-partial hazard}}$$

```
In [54]: from lifelines import CoxPHFitter
from lifelines.datasets import load_rossi

rossi_dataset = load_rossi()
rossi_dataset.head()
```

```
Out[54]:
```

	week	arrest	fin	age	race	wexp	mar	paro	prio
0	20	1	0	27	1	0	0	1	3
1	17	1	0	18	1	0	0	1	8
2	25	1	0	19	0	1	0	1	13
3	52	0	1	23	1	1	1	1	1
4	52	0	0	19	0	1	0	1	3

This data set is originally from Rossi et al. (1980), and is used as an example in Allison (1995). The data pertain to 432 convicts who were released from Maryland state prisons in the 1970s and who were followed up for one year after release. Half the released convicts were assigned at random to an experimental treatment in which they were given financial aid; half did not receive aid.

- week: week of first arrest after release or censoring; all censored observations are censored at 52 weeks.
- arrest: 1 if arrested, 0 if not arrested.
- fin: financial aid: no yes.
- age: in years at time of release.
- race: black or other.
- wexp: full-time work experience before incarceration: no or yes.
- mar: marital status at time of release: married or not married.
- paro: released on parole? no or yes.
- prio: number of convictions prior to current incarceration.

```
In [55]: cph = CoxPHFitter()  
cph.fit(rossi_dataset, duration_col='week', event_col='arrest', show_progress=True)  
  
cph.print_summary()
```

Iteration 1: norm_delta = 5.09e-01, step_size = 0.9500, log_lik = -675.38063, new
ton_decrement = 1.68e+01, seconds_since_start = 0.0

Iteration 2: norm_delta = 1.39e-01, step_size = 0.9500, log_lik = -659.79004, new
ton_decrement = 9.92e-01, seconds_since_start = 0.0

Iteration 3: norm_delta = 1.80e-02, step_size = 0.9500, log_lik = -658.76197, new
ton_decrement = 1.42e-02, seconds_since_start = 0.0

Iteration 4: norm_delta = 1.83e-04, step_size = 1.0000, log_lik = -658.74766, new
ton_decrement = 1.32e-06, seconds_since_start = 0.0

Iteration 5: norm_delta = 1.97e-08, step_size = 1.0000, log_lik = -658.74766, new
ton_decrement = 1.34e-14, seconds_since_start = 0.0

Convergence success after 5 iterations.

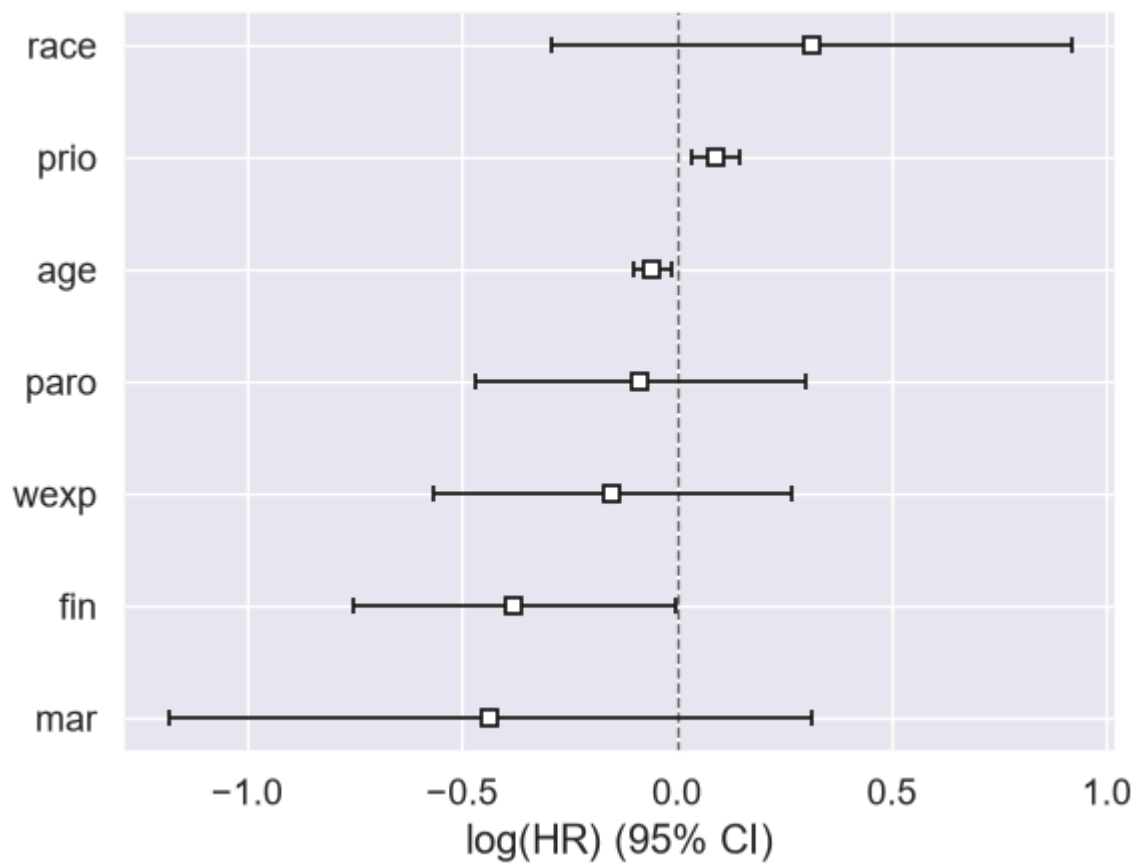
model	lifelines.CoxPHFitter									
duration col	'week'									
event col	'arrest'									
baseline estimation	breslow									
number of observations	432									
number of events observed	114									
partial log-likelihood	-658.75									
time fit was run	2026-08-28 04:43:23 UTC									

	coef	exp(coef)	se(coef)	coef lower 95%	coef upper 95%	exp(coef) lower 95%	exp(coef) upper 95%	cmp to	z	p
fin	-0.38	0.68	0.19	-0.75	-0.00	0.47	1.00	0.00	-1.98	0.05
age	-0.06	0.94	0.02	-0.10	-0.01	0.90	0.99	0.00	-2.61	0.01
race	0.31	1.37	0.31	-0.29	0.92	0.75	2.50	0.00	1.02	0.31
wexp	-0.15	0.86	0.21	-0.57	0.27	0.57	1.30	0.00	-0.71	0.48
mar	-0.43	0.65	0.38	-1.18	0.31	0.31	1.37	0.00	-1.14	0.26
paro	-0.08	0.92	0.20	-0.47	0.30	0.63	1.35	0.00	-0.43	0.66
prio	0.09	1.10	0.03	0.04	0.15	1.04	1.16	0.00	3.19	<0.005

Concordance	0.64
Partial AIC	1331.50
log-likelihood ratio test	33.27 on 7 df
-log2(p) of ll-ratio test	15.37

In [56]: `cph.plot()`

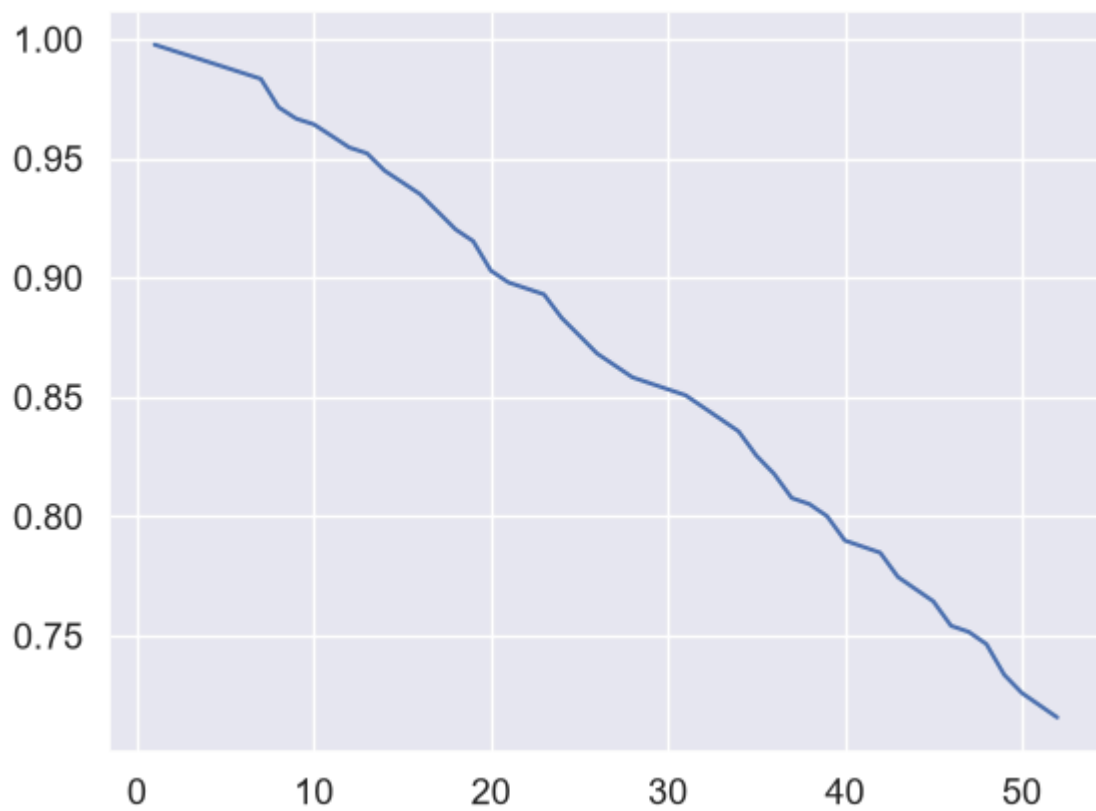
Out[56]: `<Axes: xlabel='log(HR) (95% CI)'>`



```
In [57]: X = rossi_dataset
```

```
plt.plot(cph.predict_survival_function(X[0:1]))
```

```
Out[57]: [matplotlib.lines.Line2D at 0x222d5ac73b0>]
```



Activity - Estimating Lifetime

- Use Telco Churn dataset and build a Kaplan Meier survival analysis.
- Compare lifetime between different payment methods.

```
In [58]: # =====
# Activity: Estimating Lifetime with Kaplan-Meier (Telco Churn)
# 67070501015 Naphat Utubuawong
# =====

import pandas as pd
import matplotlib.pyplot as plt
from lifelines import KaplanMeierFitter
from lifelines.statistics import multivariate_logrank_test

# 1) Load the Telco churn data
telco = pd.read_csv('Telco+Churn.csv')

# 2) Define the survival variables
#   duration (T) = tenure -> how many months the customer has stayed
#   event      (E) = Churn -> 1 if the customer left (event observed), 0 if still
T = telco['tenure']
E = (telco['Churn'] == 'Yes').astype(int)

# 3) Overall survival curve
kmf = KaplanMeierFitter()
kmf.fit(T, event_observed=E, label='All customers')

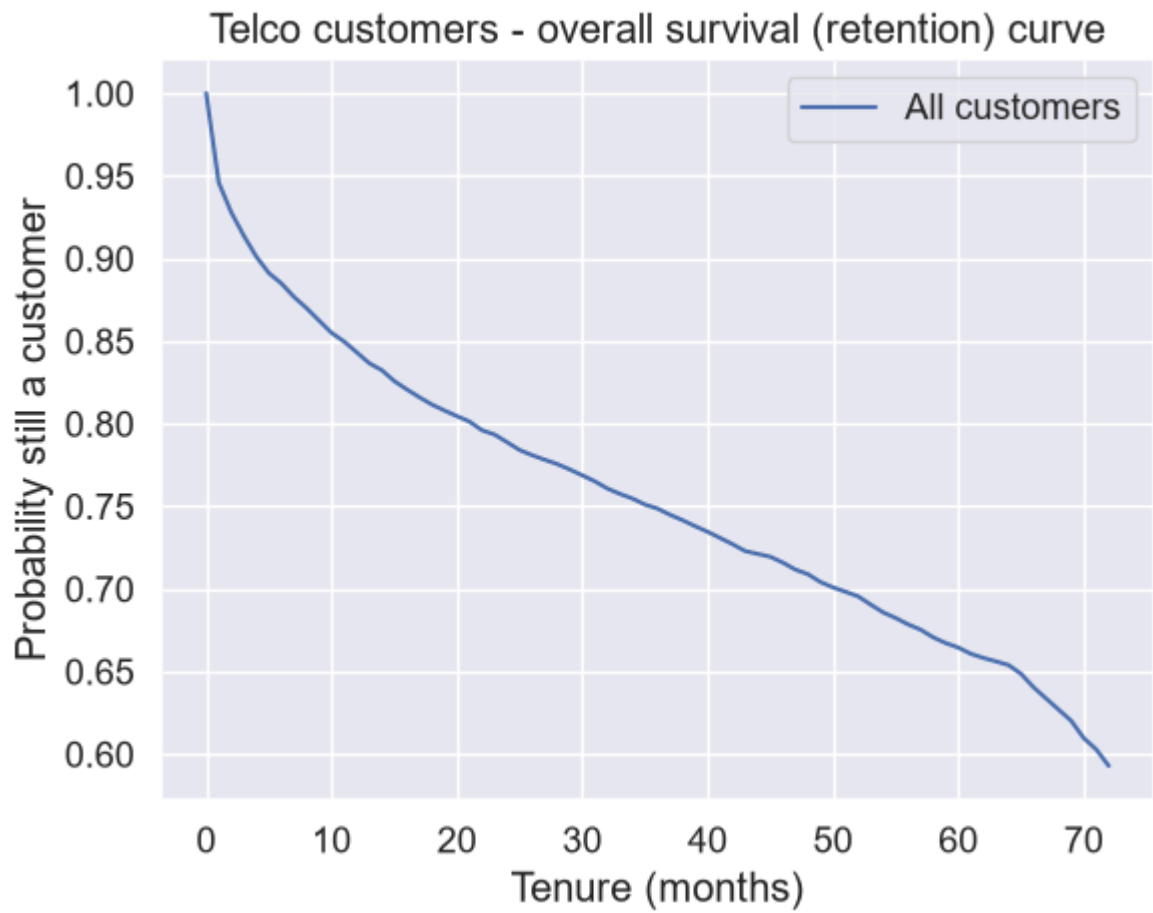
ax = plt.subplot(111)
kmf.survival_function_.plot(ax=ax)
plt.title('Telco customers - overall survival (retention) curve')
plt.xlabel('Tenure (months)')
plt.ylabel('Probability still a customer')
plt.show()

print('Overall median lifetime (months):', kmf.median_survival_time_)

# 4) Compare lifetime between different payment methods
ax = plt.subplot(111)
print('\nMedian lifetime by payment method:')
for method in telco['PaymentMethod'].unique():
    mask = telco['PaymentMethod'] == method
    kmf.fit(T[mask], event_observed=E[mask], label=method)
    kmf.plot_survival_function(ax=ax)
    print(f' {method:28s}: {kmf.median_survival_time_} months')

plt.title('Survival by payment method')
plt.xlabel('Tenure (months)')
plt.ylabel('Probability still a customer')
plt.show()

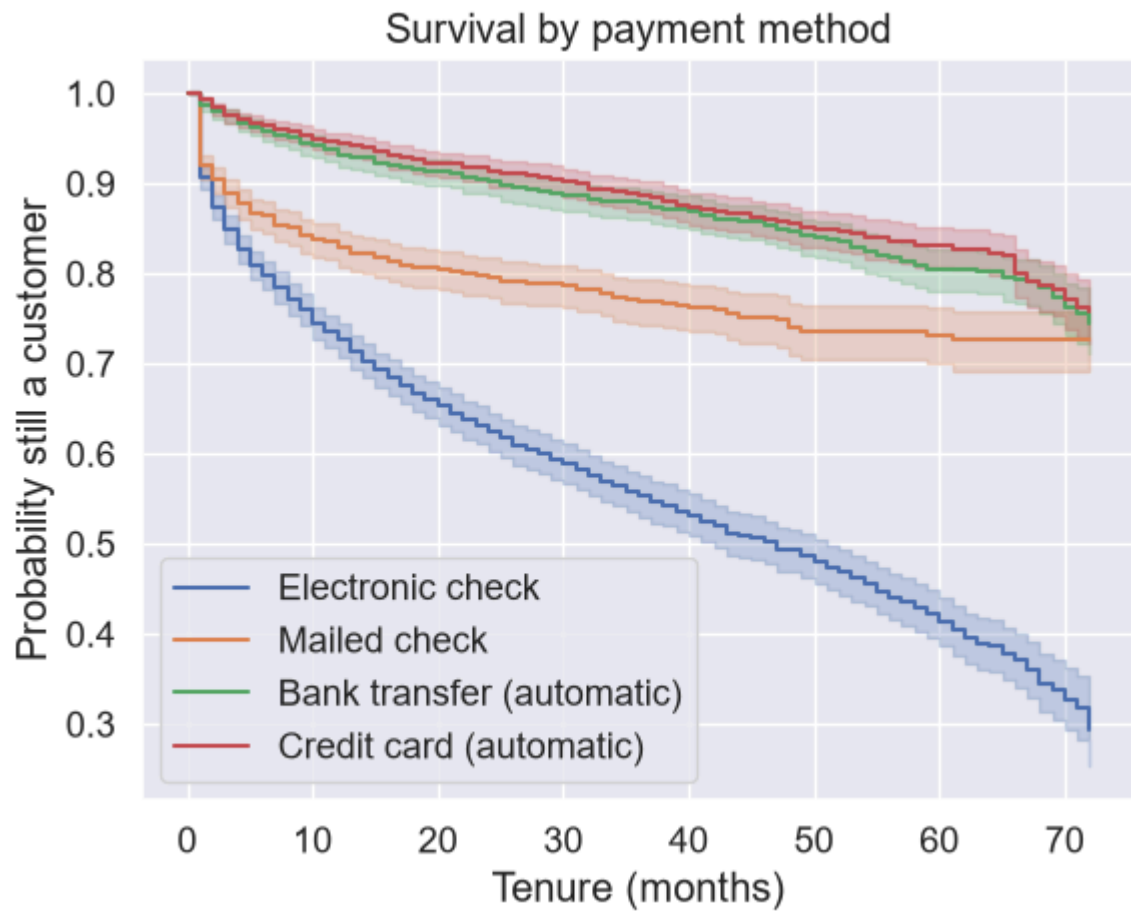
# 5) Are the differences statistically significant? (multivariate Log-rank test)
results = multivariate_logrank_test(T, telco['PaymentMethod'], E)
print('\nLog-rank test across payment methods:')
print(' test statistic =', round(results.test_statistic, 2))
print(' p-value       =', results.p_value)
```



Overall median lifetime (months): inf

Median lifetime by payment method:

Electronic check	: 47.0 months
Mailed check	: inf months
Bank transfer (automatic)	: inf months
Credit card (automatic)	: inf months



Log-rank test across payment methods:

test statistic = 865.24

p-value = 3.066529222949659e-187